

VSAM

Working with VSAM Data Sets





VSAM DATA SETS

ESDS – Entry-Sequenced Data Set

Records are stored in the order they are loaded and retrieved sequentially.

RRDS – Relative Record Data Set

Records are stored in the order they are loaded and retrieved by Relative Record Number (RRN).

KSDS – Key-Sequenced Data Set

Records are stored in order by logical key and can be browsed or accessed directly.

LDS – Linear Data Set

Processed as a byte stream. Used in z/OS system services, not in applications.

VSAM stands for Virtual Storage Access Method.

For many operational purposes, VSAM supplanted the original, more primitive access methods.

In a Platform Engineer role, you won't need to understand the internals of VSAM. You will probably have to provision environments that include VSAM data sets.

For backward compatibility, z/OS still supports the BSAM and BDAM access methods for sequential and random record access respectively, but nearly all applications use QSAM or VSAM ESDS for sequential processing, and VSAM KSDS or RRDS for random access.

VSAM LDS data sets are used for the system spooler, DB2, and other z/OS services. Applications don't use LDS, and high-level application languages like COBOL have no support for it. LDS is the only native byte-stream data set type on z/OS.

VSAM CLUSTERS

VSAM data sets are called "clusters" because they consist of more than one component, each with a separate catalog entry.

ESDS, RRDS, and LDS

- Catalog entry for the cluster
- Catalog entry for the data component

KSDS

- Catalog entry for the cluster
- Catalog entry for the data component
- Catalog entry for the index component

AIX (alternate index)

- Catalog entry for the cluster
- Catalog entry for the data component
- Catalog entry for the index component
- Catalog entry for the path component

DEFINE VSAM ESDS CLUSTER

```
//IBMUSERE JOB , 'DEFINE ESDS',  
//          CLASS=A,MSGCLASS=X,  
//          MSGLEVEL=(1,1),NOTIFY=&SYSUID  
//*****  
/* Define a VSAM ESDS  
//*****  
//DEFESDS EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSOUT DD SYSOUT=*  
//SYSIN DD *  
DELETE IBMUSER.TEST.ESDS  
IF LASTCC < 9 THEN -  
  DEFINE CLUSTER( -  
    NAME(IBMUSER.TEST.ESDS) -  
    RECORDSIZE(80 80) -  
    CYLINDERS(2 1) -  
    CISZ(4096) -  
    NONINDEXED -  
  ) -  
  DATA( -  
    NAME(IBMUSER.TEST.ESDS.DATA) -  
  )  
/*
```

We use the IDCAMS utility to define VSAM clusters. There are other ways, but this is the most common.

You can see the structure of the job is the same as for defining a GDG. The particular IDCAMS commands are different.

DEFINE CLUSTER pertains to VSAM data sets.

The DSN will be the name of the cluster entry in the catalog.

DEFINE VSAM ESDS CLUSTER

```
//IBMUSERE JOB , 'DEFINE ESDS',  
//          CLASS=A,MSGCLASS=X,  
//          MSGLEVEL=(1,1),NOTIFY=&SYSUID  
//*****  
/* Define a VSAM ESDS  
//*****  
//DEFESDS EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSOUT DD SYSOUT=*  
//SYSIN DD *  
DELETE IBMUSER.TEST.ESDS  
IF LASTCC < 9 THEN -  
  DEFINE CLUSTER( -  
    NAME(IBMUSER.TEST.ESDS) -  
    RECORDSIZE(80 80) -  
    CYLINDERS(2 1) -  
    CTS7(4096) -  
    NONINDEXED -  
  ) -  
  DATA( -  
    NAME(IBMUSER.TEST.ESDS.DATA) -  
  )  
/*
```

VSAM data sets are always treated as having variable-length logical records. For RECORDSIZE, we specify the *average* and *maximum* lengths.

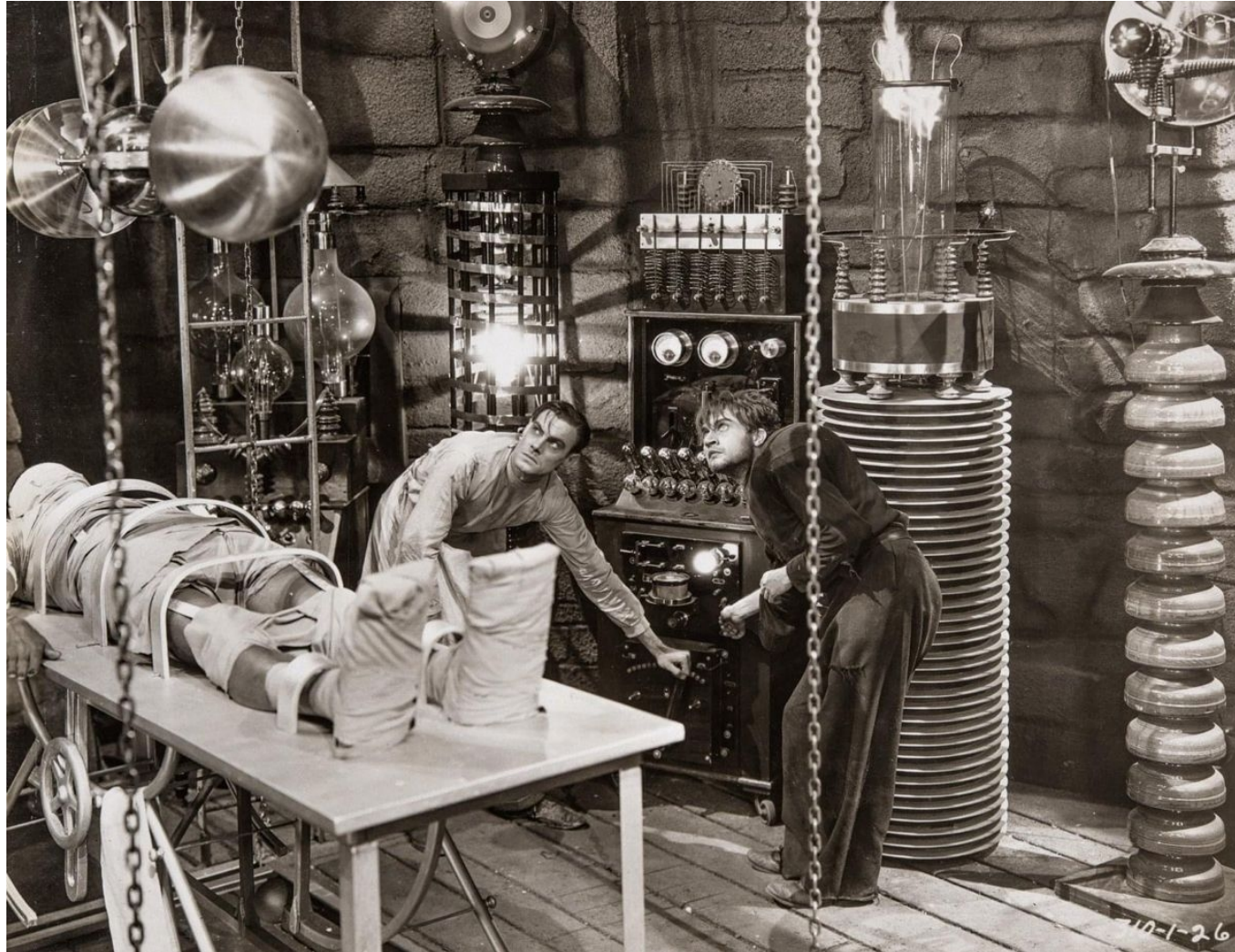
NONINDEXED is the keyword that identifies this cluster as an ESDS as opposed to any of the other VSAM data set types.

DEFINE VSAM ESDS CLUSTER

```
//IBMUSERE JOB , 'DEFINE ESDS',  
//          CLASS=A,MSGCLASS=X,  
//          MSGLEVEL=(1,1),NOTIFY=&SYSUID  
//*****  
/* Define a VSAM ESDS  
//*****  
//DEFESDS EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSOUT DD SYSOUT=*  
//SYSIN DD *  
DELETE IBMUSER.TEST.ESDS  
IF LASTCC < 9 THEN -  
  DEFINE CLUSTER( -  
    NAME(IBMUSER.TEST.ESDS) -  
    RECORDSIZE(80 80) -  
    CYLINDERS(2 1) -  
    CISZ(4096) -  
    NONINDEXED -  
  ) -  
  DATAC -  
    NAME(IBMUSER.TEST.ESDS.DATA) -  
  )  
/*
```

The name of the data component will be the same as the name of the cluster with ".DATA" appended.

LAB Z/OS 19: DEFINE A VSAM KSDS CLUSTER



CAN YOU DEFINE CLUSTERS WITH TSO?

Most anything you can do with JCL you can also do from the TSO READY prompt. Sometimes a single command is quicker than writing JCL to do the same task.

You can run any IDCAMS command from the TSO READY prompt. The TSO command consists of the same content as the SYSIN data set in an IDCAMS batch job. However, this is *not* a case when it is simpler than writing the JCL.

The first page of help for the DEFINE CLUSTER command is shown at right. When issuing TSO commands using an off-platform tool like Ansible, you have to code the entire command on one line. IDCAMS commands can be very long. It would be difficult to maintain or even to read/understand such a command coded as a single line.

In that situation, the recommendation is to keep the JCL for IDCAMS on the Ansible control node and transfer it to z/OS nodes for execution *via* playbooks.

```
READY
help defcl

FUNCTION -
  THE DEFINE COMMAND CAN BE USED TO DEFINE A CATALOG ENTRY FOR AN
  INDEXED, NONINDEXED, OR NUMBERED (RELATIVE RECORD) CLUSTER.

SYNTAX -
  DEFINE CLUSTER
  ( NAME('ENTRYNAME')
    FILE('DNAME')
    STORAGECLASS('CLASSNAME')
    MANAGEMENTCLASS('CLASSNAME')
    DATACLASS('CLASSNAME')
    VOLUMES('VOLSER' ...)
    TRACKS('PRIMARY' 'SECONDARY') |
      CYLINDERS('PRIMARY' 'SECONDARY') |
      RECORDS('PRIMARY' 'SECONDARY') |
      KILOBYTES('PRIMARY' 'SECONDARY') |
      MEGABYTES('PRIMARY' 'SECONDARY')
    RECORDSIZE('AVERAGE' 'MAXIMUM')
    FREESPACE('CIPERCENT' 'CAPERCENT')
    SPANNED | NONSPANNED
    KEYRANGES(('LOWKEY' 'HIGHKEY') ...)
    ORDERED | UNORDERED
    BUFFERSPACE('SIZE')
    CONTROLINTERVALSIZE('SIZE')
    OWNER('OWNERID')
    TO('DATE') | FOR('DAYS')
    SHAREOPTIONS('CROSSREGION' 'CROSSSYSTEM')
    ERASE | NOERASE
    EXCEPTIONEXIT('MNAME')
    SPEED | RECOVERY
    WRITECHECK | NOWRITECHECK
    INDEXED | NONINDEXED | NUMBERED
    LOG(UNDO|NONE|ALL)
    LOGSTREAMID('NAME')
    RLSQUIESCE | RLSENABLE
    BWO(TYPECICS|TYPEIMS|NO)
    FRLOG(NONE|REDO)
    KEYS('LENGTH' 'OFFSET')
    REUSE | NOREUSE

***
```

WHAT YOU KNOW AND WHAT YOU CAN DO

- You can define VSAM assets using JCL

WHAT'S NEXT

- REXX, TSO, and batch